

CStats - Lab #9: Convolutional Neural Networks

May 5, 2026

Exercise 1.

A grayscale image can be treated as a matrix of numbers. For example, the following 4×4 image has a bright 2×2 square in the middle:

$$X = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

1. How many input numbers does this image contain?
2. A color image of size 100×100 has 3 channels: red, green, and blue. How many input numbers does it contain?
3. Suppose we feed that $100 \times 100 \times 3$ image into a fully connected hidden layer with 1000 neurons. How many weights are in this first layer, ignoring biases?
4. Why might this be a strange first layer for image processing? Give an intuitive answer, not just a parameter count.

Exercise 2.

This exercise introduces the basic convolution operation. Consider the input image

$$X = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix}$$

and the 2×2 filter

$$K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

We use stride 1 and no padding, so the filter only sits where it fits completely inside the image. The figure for this exercise is shown at the end of the lab under **Exercise 2**.

1. Compute the top-left output value. That is, place K on the top-left 2×2 window of X and take the entry-by-entry product and sum.
2. Compute the full output feature map.
3. What kind of tiny pattern is this filter looking for?

Exercise 3.

A CNN does two important things: it looks locally, and it reuses the same filter in many locations.

Imagine an 8×8 grayscale image and a detector that only needs to inspect a 3×3 window. The figure for this exercise is shown at the end of the lab under **Exercise 3**.

1. With no padding and stride 1, how many different 3×3 windows are there in the 8×8 image?
2. Suppose we make a locally connected layer with one neuron per 3×3 window, but every neuron has its own separate weights. How many parameters are needed, including one bias per neuron?
3. Suppose instead we use one convolutional filter shared across all locations. How many parameters are needed, including one bias?
4. Suppose we use 64 different shared 3×3 filters. What is the output shape? How many trainable parameters are in this convolutional layer?
5. Explain why parameter sharing is a good bias for image models.

Exercise 4.

Now we use filters as simple pattern detectors. Consider the vertical-line image

$$X_v = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$

and two 3×3 filters:

$$K_v = \begin{bmatrix} -1 & 2 & -1 \\ -1 & 2 & -1 \\ -1 & 2 & -1 \end{bmatrix}, \quad K_h = \begin{bmatrix} -1 & -1 & -1 \\ 2 & 2 & 2 \\ -1 & -1 & -1 \end{bmatrix}.$$

The figure for this exercise is shown at the end of the lab under **Exercise 4**.

1. Place each filter on the center 3×3 window of X_v . Compute the two outputs.
2. Which filter responds more strongly to the vertical line?
3. If the input instead had a horizontal line through the center row, what would you expect to happen?
4. In a real CNN, would we usually hand-code K_v and K_h ? Explain.

Exercise 5.

Stride controls how far the filter jumps each time it moves. Padding adds artificial border pixels, usually zeros. For a square input of width W , a square filter of width F , padding P , and stride S , the spatial output size is

$$\frac{W - F + 2P}{S} + 1,$$

when this is an integer. The figure for this exercise is shown at the end of the lab under **Exercise 5**.

1. A 7×7 image is convolved with one 3×3 filter, no padding, stride 1. What is the output size?
2. Same image and filter, no padding, stride 2. What is the output size? What changed intuitively?
3. Same image and filter, padding 1, stride 1. What is the output size? Why is padding useful?
4. Apply 2×2 max pooling with stride 2 to the feature map

$$A = \begin{bmatrix} 3 & -1 & -3 & -1 \\ -3 & 1 & 0 & -3 \\ -3 & -3 & 0 & 1 \\ 3 & -2 & -2 & -1 \end{bmatrix}.$$

Compute the pooled output.

5. Fill in the shapes of this small CNN:

$28 \times 28 \times 1$
 $\rightarrow$ Conv(8 filters, 3×3 , $S = 1$, $P = 1$)
 $\rightarrow$ MaxPool(2×2 , $S = 2$)
 $\rightarrow$ Conv(16 filters, 3×3 , $S = 1$, $P = 1$)
 $\rightarrow$ MaxPool(2×2 , $S = 2$)
 $\rightarrow$ Flatten $\rightarrow$ Dense(10).

Exercise 6.

A convolutional layer is usually followed by a nonlinearity. One common choice is ReLU:

$$\text{ReLU}(z) = \max(0, z), \quad H = \text{ReLU}(X * K + b).$$

The convolution creates a feature map of scores, then the nonlinearity is applied to that feature map entry by entry.

- (a) Compute $\text{ReLU}(Z)$ for

$$Z = \begin{bmatrix} 2 & -1 \\ 0 & 3 \end{bmatrix}.$$

- (b) Now see why nonlinearities are not just decoration. Suppose we stack two 1D convolutions with no biases and no nonlinearities. The first filter is $K_1 = [1, 1]$, and the second filter is $K_2 = [1, -1]$. For input $[x_1, x_2, x_3]$, using valid convolutions, the first layer gives

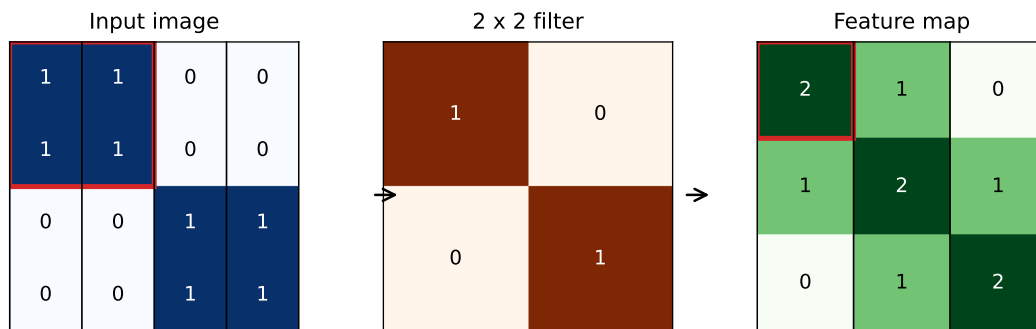
$$h_1 = x_1 + x_2, \quad h_2 = x_2 + x_3.$$

The second layer gives $y = h_1 - h_2$. Simplify y . What single length-3 filter would give the same output directly from $[x_1, x_2, x_3]$?

- (c) Explain in words why deep CNNs insert a nonlinearity such as ReLU after each convolution.

Figures

Exercise 2: What is a convolution?



Exercise 3: Local fields and parameter sharing

Same small pattern, two locations

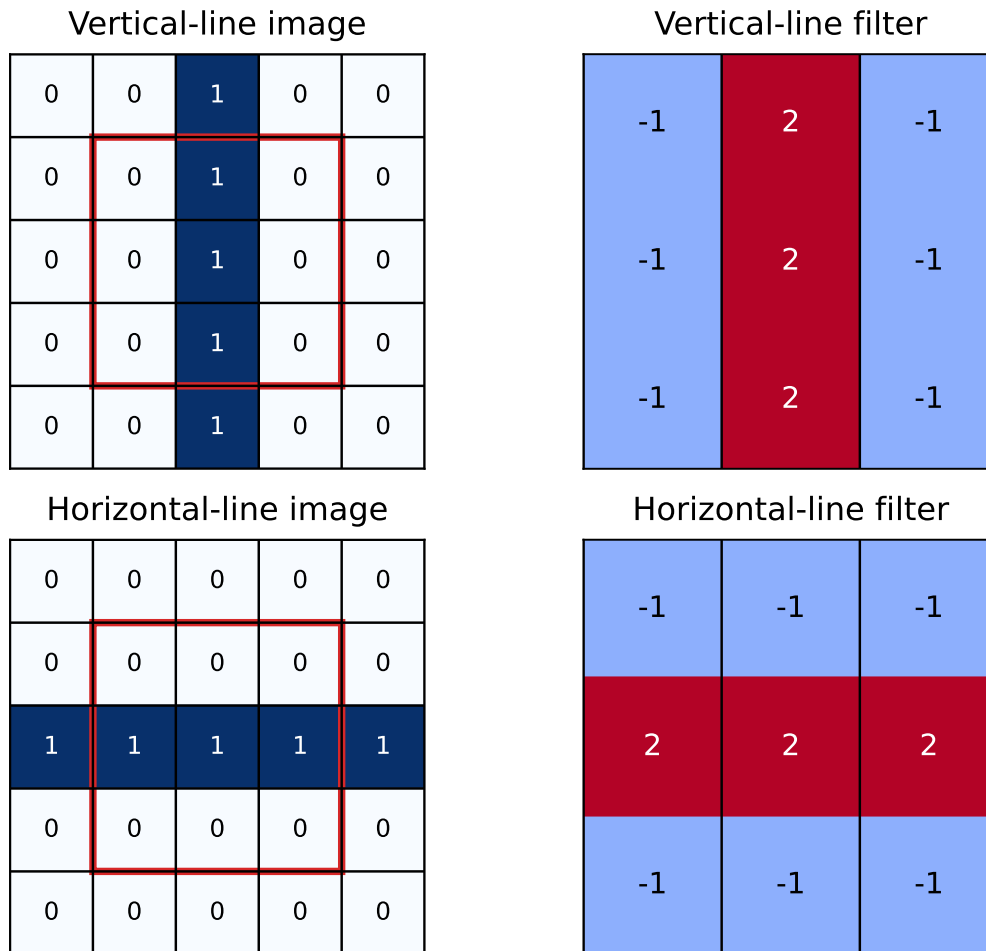
0	0	0	0	0	0	0	0
0	0	1	0	0	0	0	0
0	1	1	1	0	0	0	0
0	0	1	0	0	0	0	0
0	0	0	0	0	1	0	0
0	0	0	0	1	1	1	0
0	0	0	0	0	1	0	0
0	0	0	0	0	0	0	0

One shared filter

0	1	0
1	1	1
0	1	0

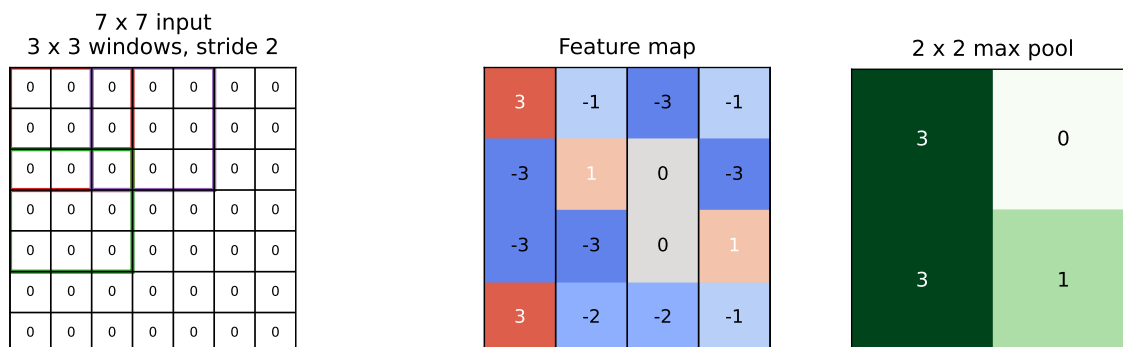
A convolution reuses the same weights at every spatial position.

Exercise 4: Simple filters as pattern detectors



A high dot product means the local window resembles the filter.

Exercise 5: Stride, padding, pooling, and the full CNN



Stride skips positions. Max pooling keeps the strongest response in each small block.